iBanking

Hệ thống Theo dõi và Thanh toán Học phí

Auth Service
Port 8001 · JWT, số dư

Tuition Service
Port 8002 · học phí, gạch nợ

Payment Service
Port 8003 · Orchestrator, FSM, Saga

Notification Service
Port 8004 · OTP, email

Tổng quan hệ thống

NGHIỆP VỤ CỐT LÕI

Theo dõi & thanh toán học phí nhanh chóng

Sinh viên tra cứu khoản học phí theo MSSV và thanh toán bằng số dư tài khoản iBanking, mỗi giao dịch được xác nhận bằng OTP gửi qua email.

4
service độc lập

1
API Gateway

4
database riêng

5 phút
hiệu lực OTP

Đăng nhập & xác thực

JWT hạn 2 giờ; mọi API công khai đều kiểm tra token, hết hạn thì tự đăng xuất.

Tra cứu học phí

Nhập MSSV để xem thông tin sinh viên và các khoản phí (Chưa / Đã thanh toán).

Thanh toán 2 bước

Khởi tạo giao dịch → nhập OTP email → gạch nợ học phí + trừ tiền. Có hủy và gửi lại OTP.

Biên lai & lịch sử

Gửi email biên lai sau khi thành công; trang lịch sử giao dịch lọc theo trạng thái.

Các Service chính

API Gateway

nginx · Port 8000

Điểm vào duy nhất của frontend: định tuyến /api/auth, /api/tuition, /api/payment; rate limit; gắn x-request-id; chặn /internal/*.

DỮ LIỆU SỞ HỮU
Không có dữ liệu

Auth Service

Port 8001

Đăng nhập, ký & kiểm tra JWT; quản lý số dư và trừ tiền an toàn (khóa dòng, chống Double Spending).

DỮ LIỆU SỞ HỮU
auth_db · users

Tuition Service

Port 8002

Tra cứu sinh viên & học phí; gạch nợ có điều kiện (chống Double Payment); hoàn tác khi Saga bù hoàn.

DỮ LIỆU SỞ HỮU
tuition_db · students, tuition_fees

Payment Service

Port 8003 · Orchestrator

Điều phối giao dịch: FSM, Saga, Idempotency-Key, lịch sử. Bên duy nhất gọi các service khác.

DỮ LIỆU SỞ HỮU
payment_db · transactions

Notification Service

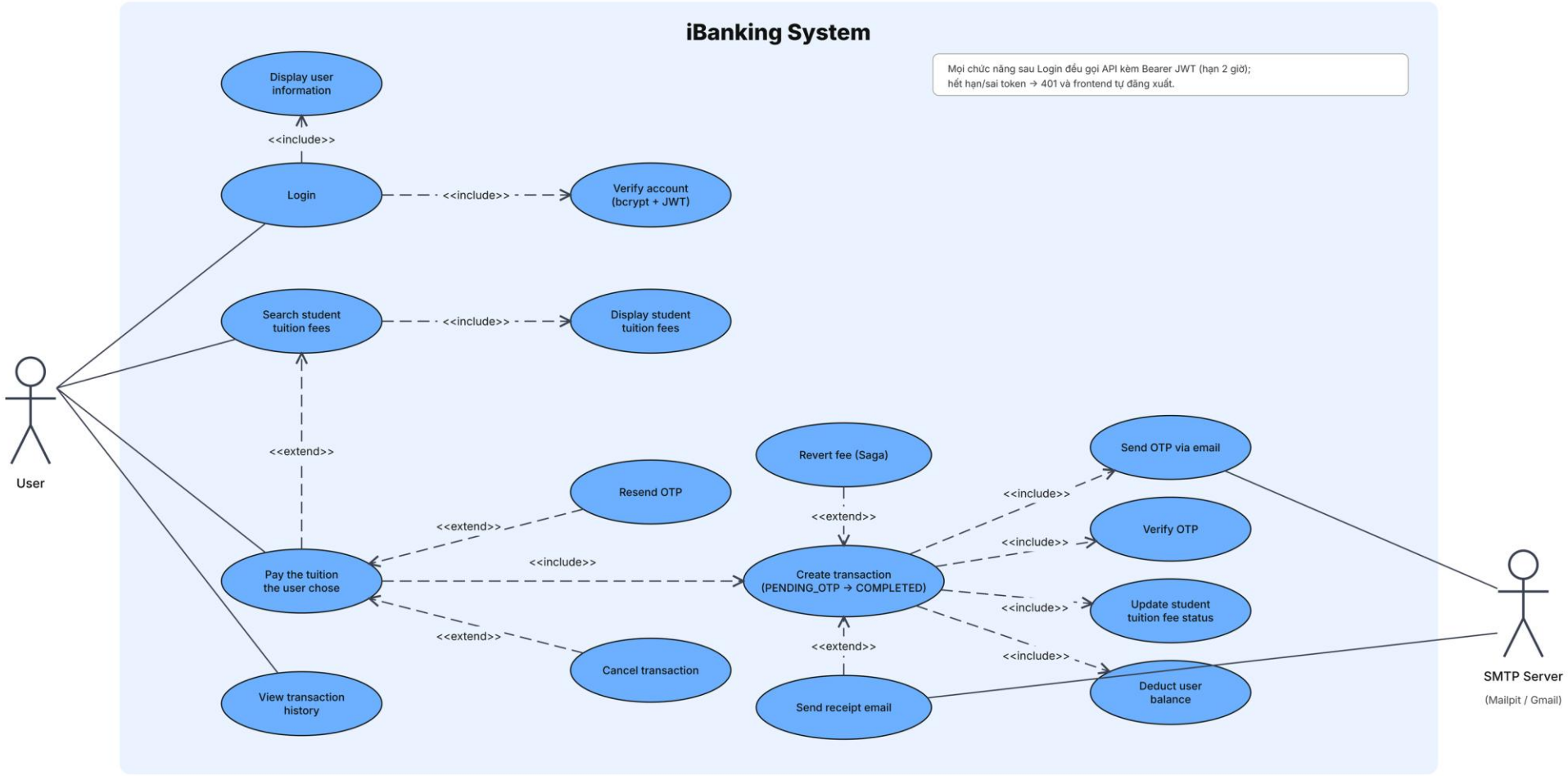
Port 8004

Sinh & xác thực OTP (6 số, 5 phút, dùng 1 lần); gửi email OTP và biên lai qua SMTP.

DỮ LIỆU SỞ HỮU
notification_db · otps

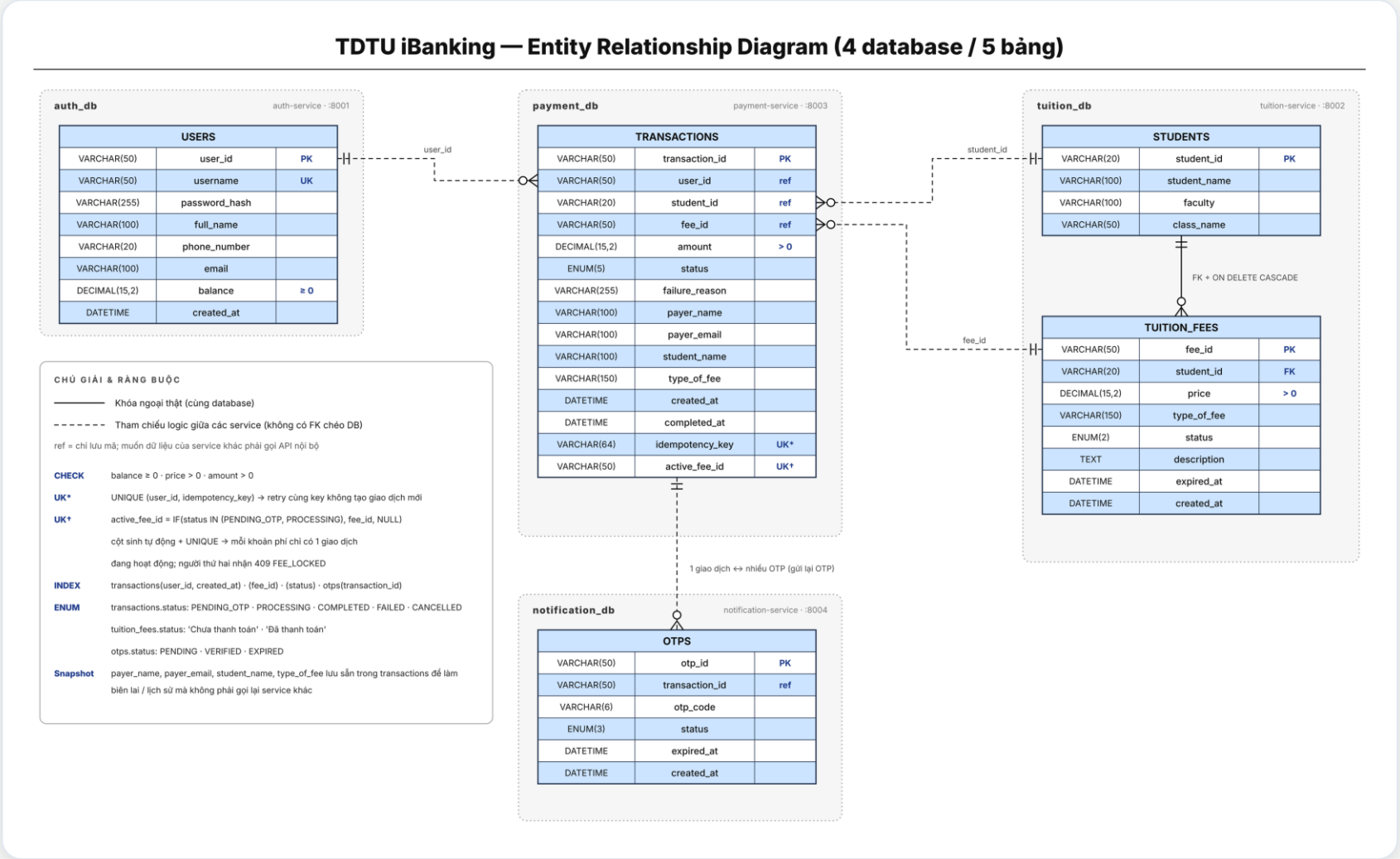
Đồ thị gọi: payment → auth · payment → tuition · payment → notification. Service chỉ truy vấn bảng của chính nó; cần dữ liệu khác phải gọi API.

Use Case Diagram



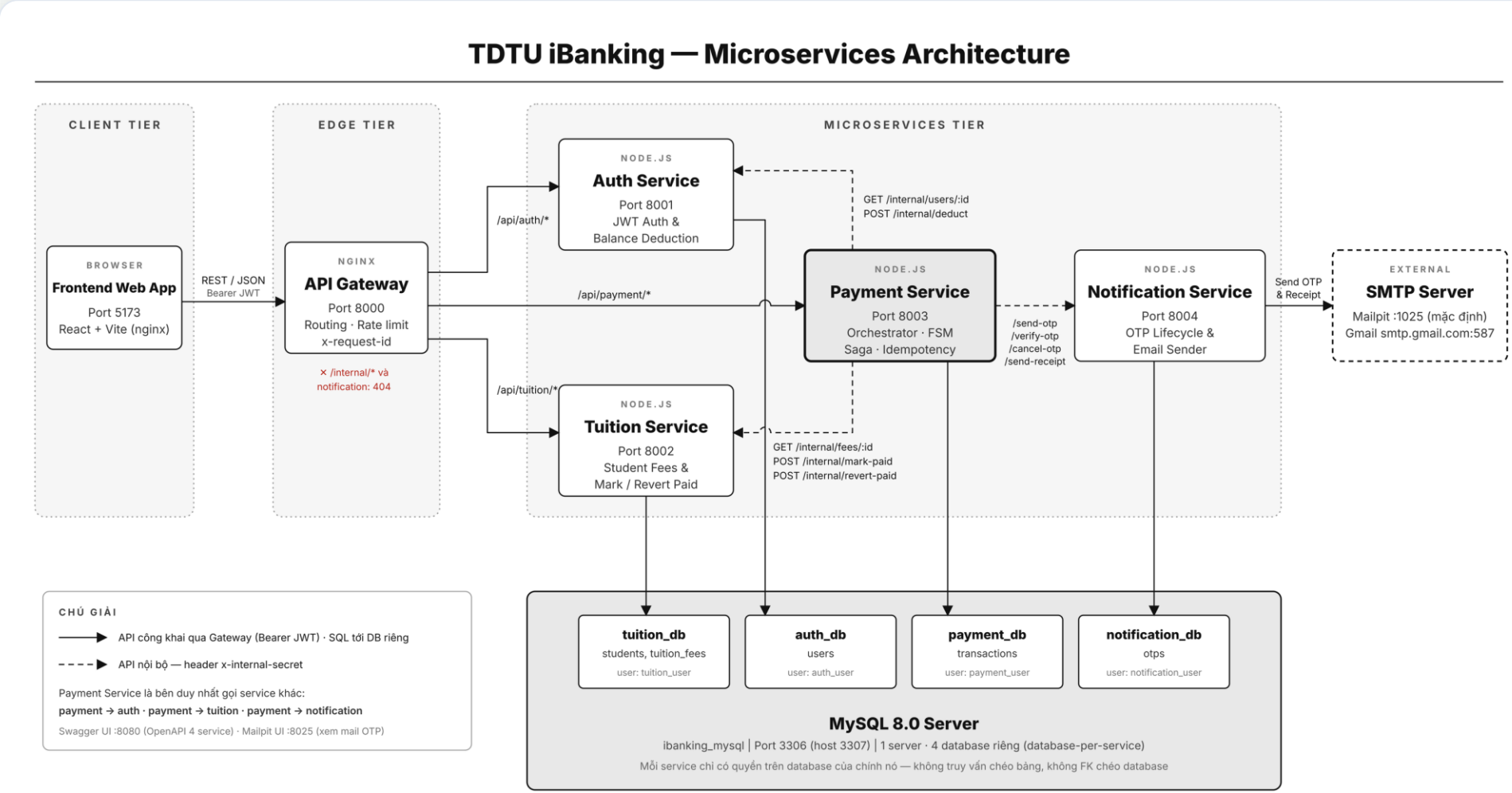
Người dùng thao tác 6 chức năng; SMTP Server là actor phụ nhận email OTP và biên lai.

Sơ đồ Thực thể Liên kết (ERD)



Database-per-service: 4 database, không có khóa ngoại chéo database; liên kết giữa các service là tham chiếu logic qua API.

Kiến trúc Microservices



Frontend chỉ gọi API Gateway; Payment Service (Orchestrator) là bên duy nhất gọi Auth, Tuition và Notification.

Giao tiếp giữa các Service & bảo mật

JWT cho API công khai

Frontend gửi Authorization: Bearer <token> (HS256, hạn 2 giờ). Payment lấy user_id từ token, không tin body; sai chủ giao dịch → 403 FORBIDDEN.

x-internal-secret cho API nội bộ

/internal/* của auth, tuition và /send-otp, /verify-otp, /cancel-otp, /send-receipt của notification. Thiếu/sai → 401 INVALID_INTERNAL_SECRET.

API Gateway (nginx)

Định tuyến theo tiền tố; không công khai /internal/* và notification (404). Rate limit: đăng nhập 30 lần/phút/IP, API chung 20 req/giây/IP (429).

Truy vết & Idempotency

x-request-id đi xuyên Gateway → 4 service, ghi vào mỗi dòng log JSON. Header Idempotency-Key + UNIQUE (user_id, key) chống tạo giao dịch trùng khi retry.

Payment gọi service khác qua callService(): timeout 10 giây, không bao giờ ném lỗi mà trả { ok, status, data, networkError } để orchestrator tự quyết định tiếp tục, FAILED hay chạy Saga. Lỗi mạng ở mark-paid không revert mù mà ghi log để đối soát.

Xác thực OTP qua Email

6 chữ số, sinh bằng CSPRNG
crypto.randomInt; không trùng OTP nào đang PENDING còn hạn.

Hiệu lực 5 phút, gắn 1 giao dịch
Lưu otps (PENDING) với expired_at = NOW() + 300 giây theo transaction_id.

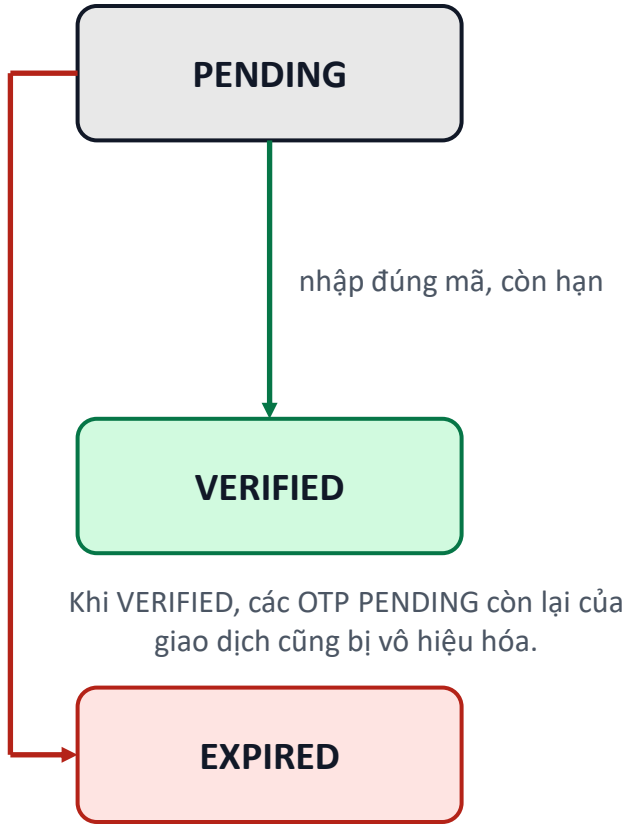
Dùng một lần
verify-otp: SELECT … FOR UPDATE + UPDATE … WHERE status = 'PENDING' → VERIFIED.

Gửi lại = OTP mới
OTP cũ chuyển EXPIRED; GET_LOCK tuần tự hóa gửi/gửi lại của một giao dịch.

Gửi email sau khi COMMIT
Không giữ khóa DB khi chờ SMTP; gửi lỗi thì OTP vừa tạo bị hủy.

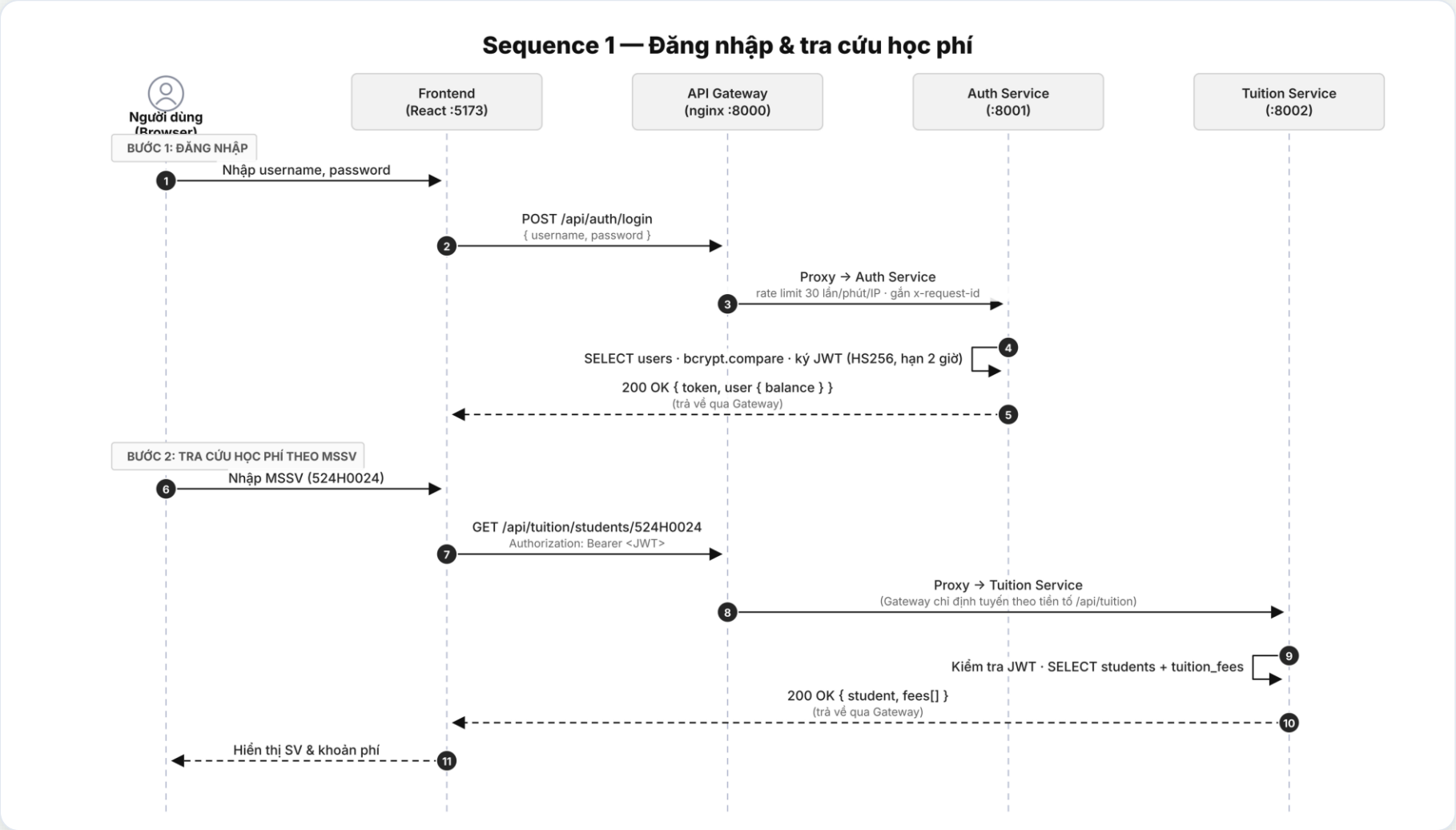
Sai hoặc hết hạn → FAILED
Cũng là giới hạn số lần đoán; người dùng phải tạo giao dịch mới.

VÒNG ĐỜI OTP (notification_db.otps)



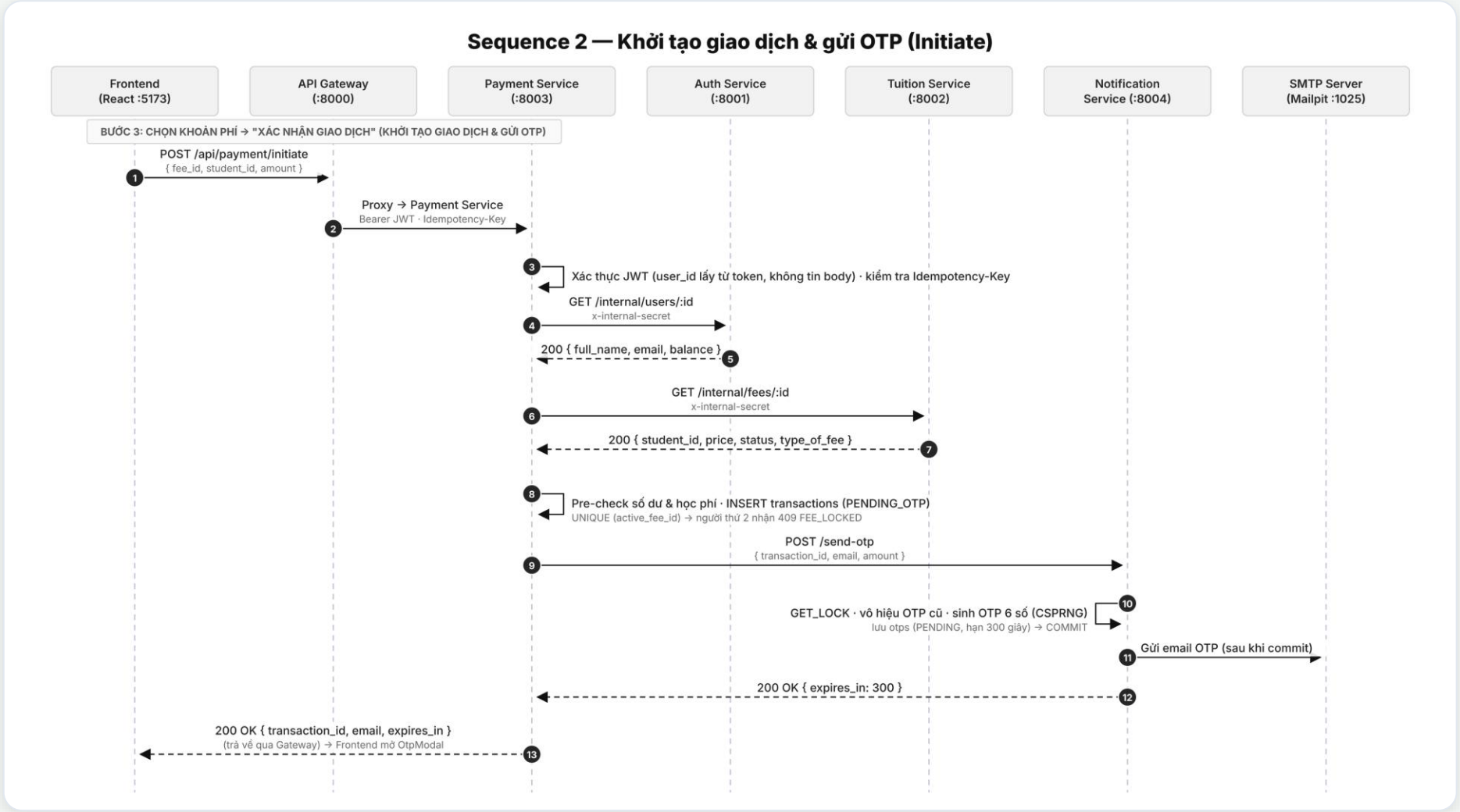
quá 5 phút · có OTP mới · hủy giao dịch

Sequence (1/4): Đăng nhập & tra cứu học phí



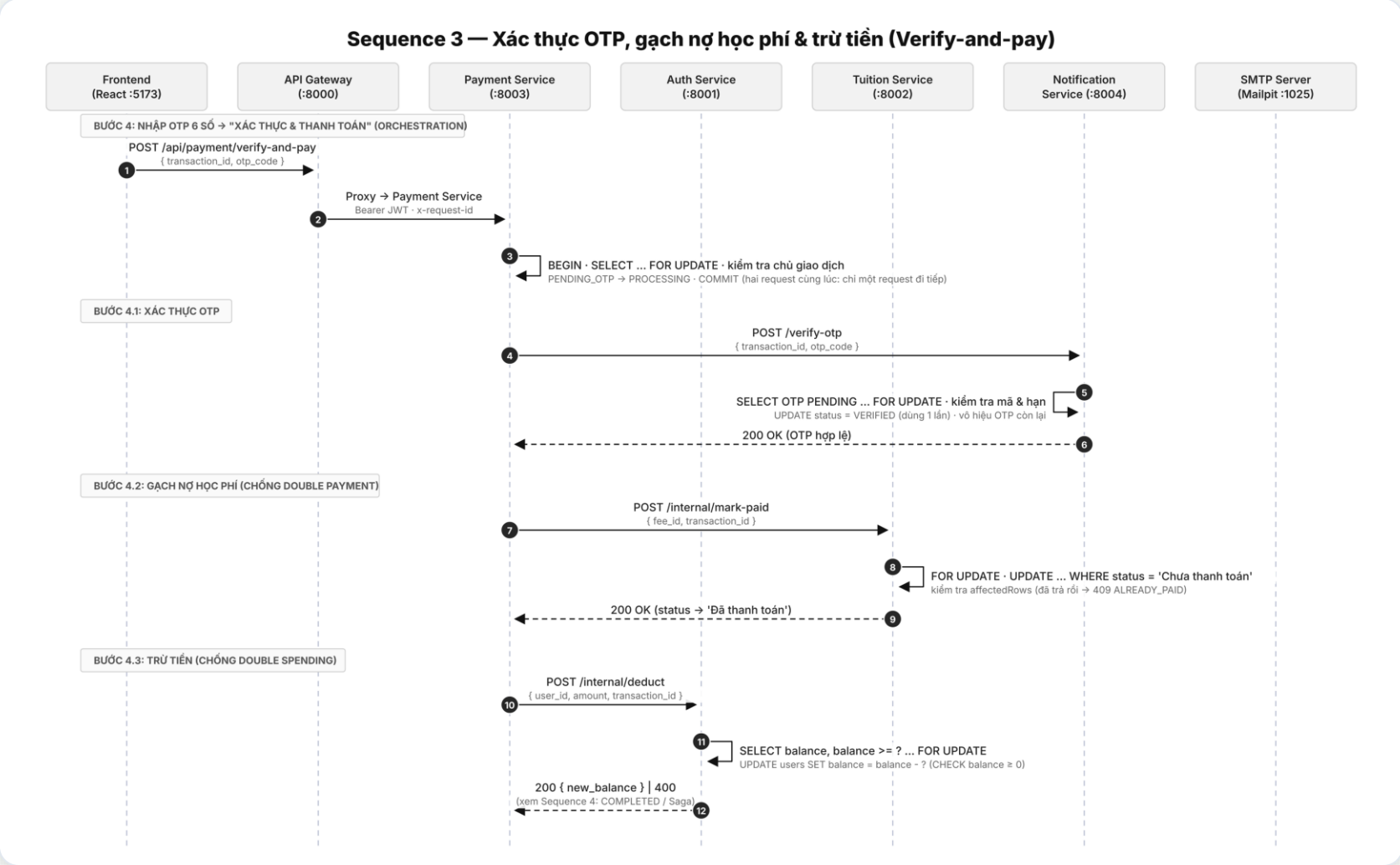
Mọi request của frontend đi qua API Gateway; sau đăng nhập, frontend gắn Bearer JWT vào mỗi lời gọi.

Sequence (2/4): Khởi tạo giao dịch & gửi OTP



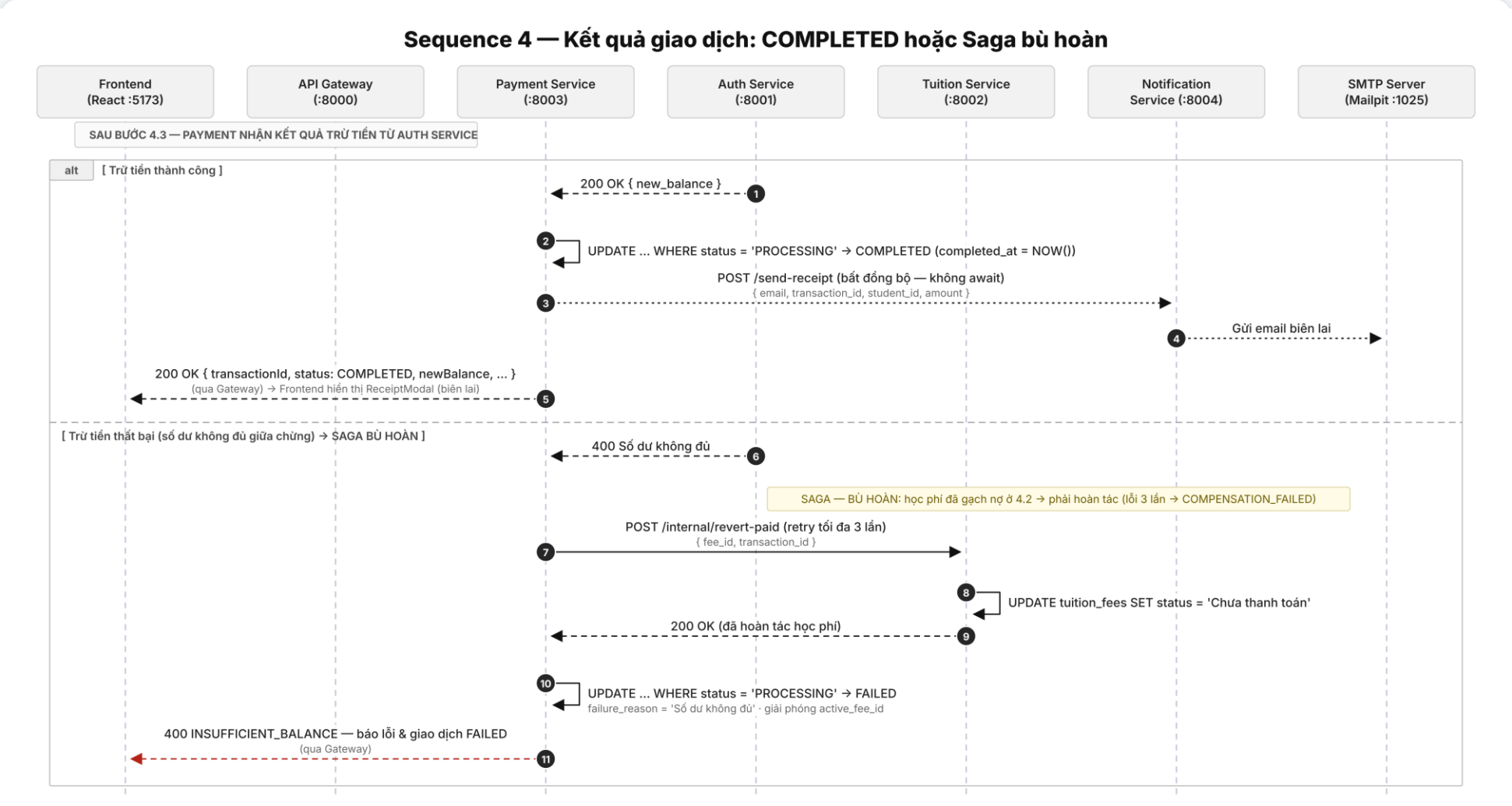
Payment pre-check qua API của Auth và Tuition, tạo giao dịch PENDING_OTP rồi nhờ Notification sinh và gửi OTP.

Sequence (3/4): Xác thực OTP, gạch nợ & trừ tiền



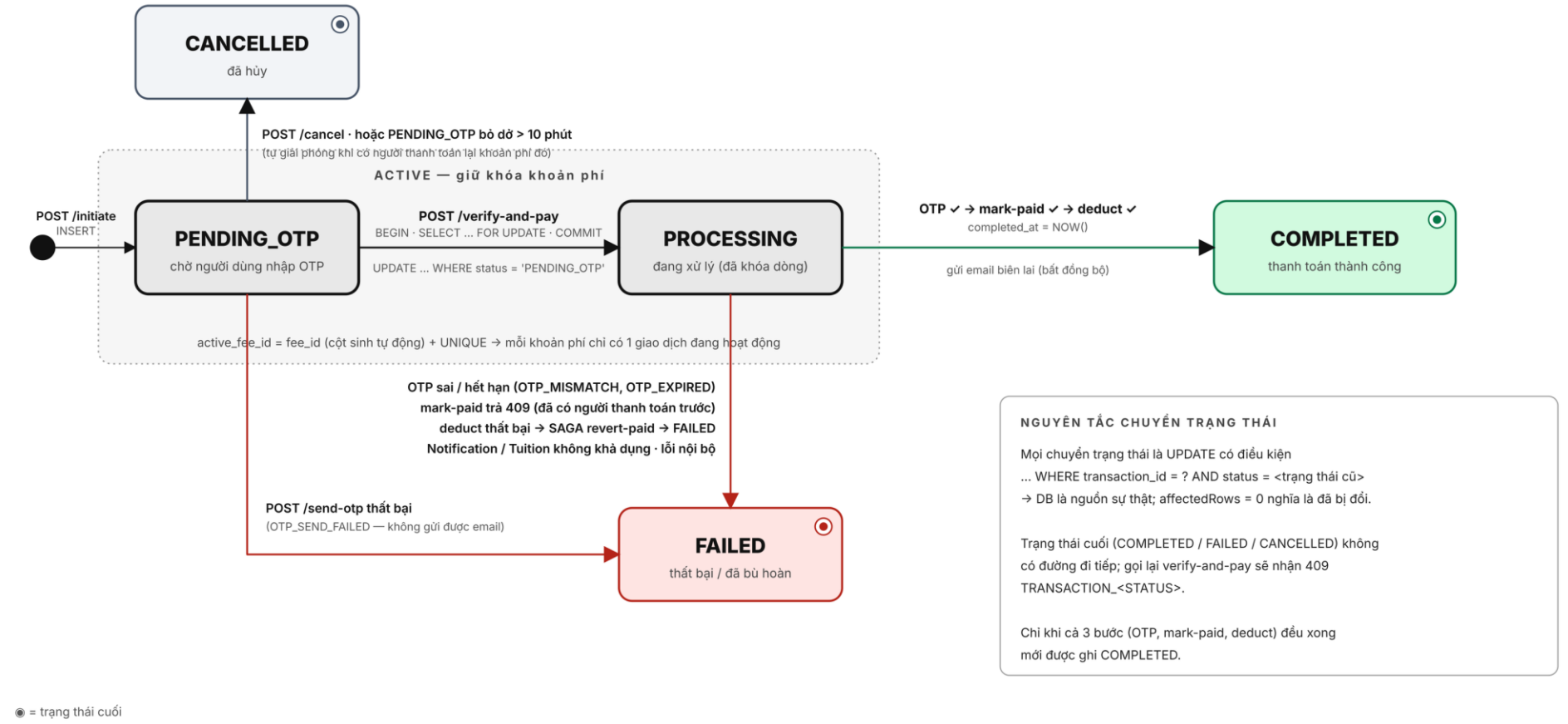
Payment khóa dòng giao dịch, chuyển PROCESSING rồi lần lượt: verify OTP → mark-paid (Tuition) → deduct (Auth).

Sequence (4/4): COMPLETED hoặc Saga bù hoàn



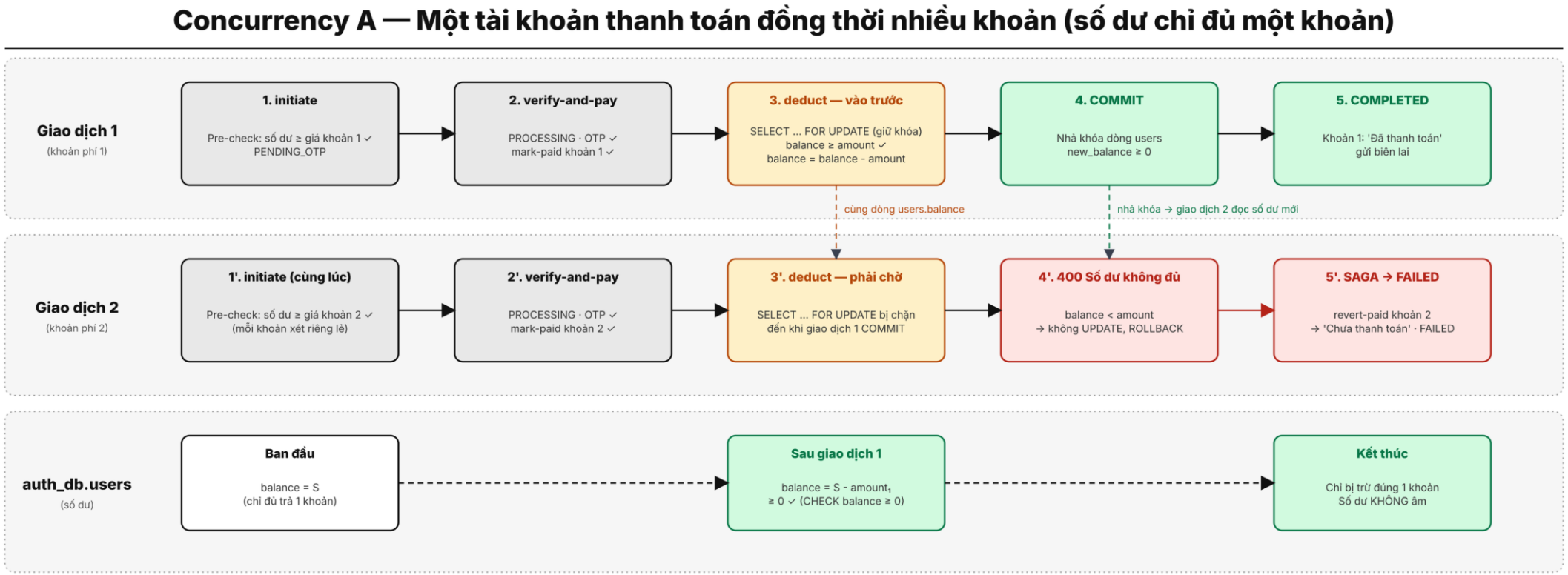
Trừ tiền lỗi sau khi đã gạch nợ → Payment gọi revert-paid (retry 3 lần) để trả học phí về "Chưa thanh toán", giao dịch FAILED.

Transaction: Máy trạng thái (FSM) & Saga



Mọi chuyển trạng thái là UPDATE có điều kiện theo trạng thái cũ; PENDING_OTP và PROCESSING giữ khóa khoản phí qua active_fee_id.

Concurrency A: Một tài khoản, nhiều khoản phí cùng lúc



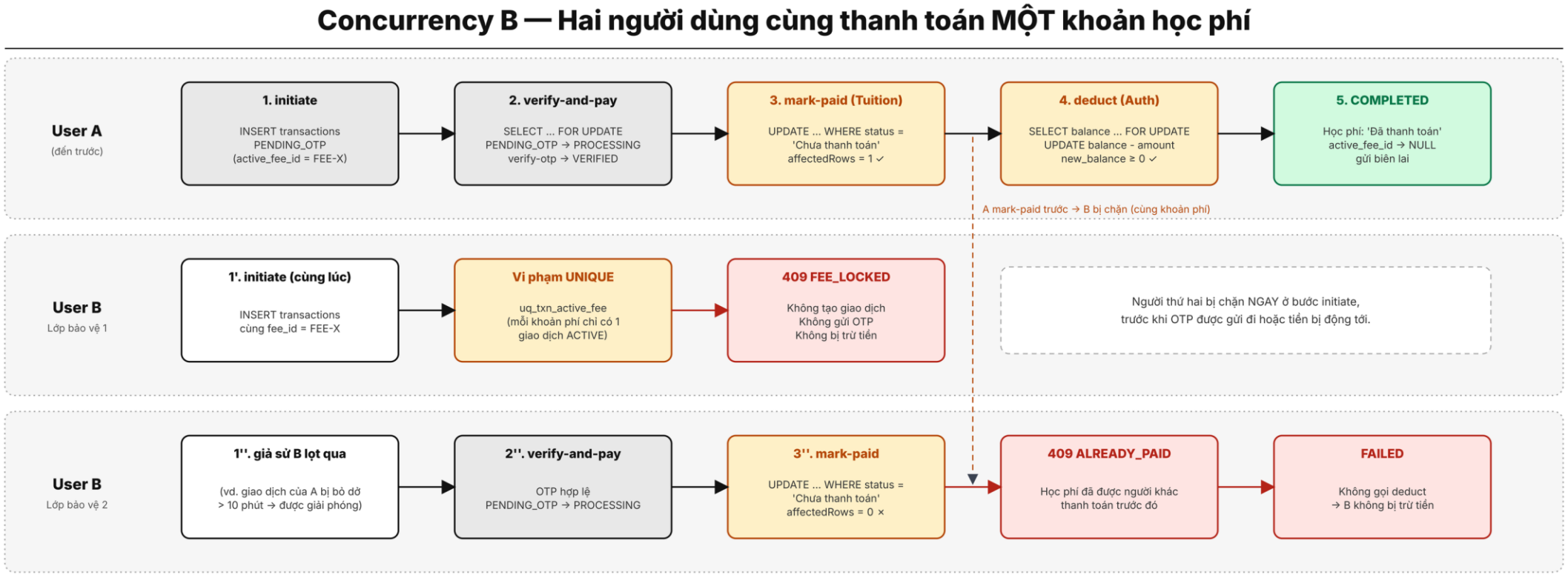
KẾT QUẢ — chống Double Spending

auth-service: SELECT balance, balance >= ? ... FOR UPDATE rồi UPDATE users SET balance = balance - ? trong cùng một transaction → các lần trừ tiền xếp hàng tuần tự.

Giao dịch thua nhận 400 INSUFFICIENT_BALANCE → payment chạy Saga revert-paid (hoàn tác gạch nợ khoản 2) → FAILED; ràng buộc CHECK (balance ≥ 0) là lớp bảo vệ cuối.

Kết quả: đúng 1 giao dịch thành công, số dư không âm, khoản phí của giao dịch thua trở lại 'Chưa thanh toán'.

Concurrency B: Nhiều tài khoản cùng trả một khoản phí



KẾT QUẢ — hai lớp bảo vệ (defense in depth)

Lớp 1 (payment-service): UNIQUE (active_fee_id) trên transactions → chặn người thứ hai ngay lúc khởi tạo (409 FEE_LOCKED).

Lớp 2 (tuition-service): SELECT ... FOR UPDATE + UPDATE ... WHERE status = 'Chưa thanh toán', kiểm tra affectedRows → chỉ 1 request gạch nợ thành công.

Kết quả: đúng 1 giao dịch COMPLETED, khoản phí chỉ bị trừ tiền một lần, người còn lại không mất tiền (không có Double Payment).

Tổng hợp cơ chế chống đồng thời

Rủi ro	Cơ chế	Nơi cài đặt
Double Spending: 1 tài khoản, nhiều giao dịch	SELECT balance … FOR UPDATE + UPDATE balance = balance - ?; CHECK (balance ≥ 0)	auth · /internal/deduct
Double Payment: nhiều người, 1 khoản phí	UPDATE … WHERE status = 'Chưa thanh toán' + kiểm tra affectedRows	tuition · /internal/mark-paid
Hai giao dịch giữ cùng một khoản phí	UNIQUE (active_fee_id) → người thứ hai nhận 409 FEE_LOCKED	payment_db · transactions
Bấm xác nhận 2 lần / verify-cancel song song	SELECT … FOR UPDATE trên dòng giao dịch; PENDING_OTP → PROCESSING nguyên tử	payment · verify-and-pay, cancel
OTP dùng hai lần / gửi lại song song	verify FOR UPDATE + UPDATE WHERE status = 'PENDING'; GET_LOCK khi gửi OTP	notification · /verify-otp, /send-otp
Retry khi mất mạng	Idempotency-Key + UNIQUE (user_id, idempotency_key)	payment · /initiate
Lỗi giữa chừng sau khi gạch nợ	Saga revert-paid (retry 3 lần); thất bại → COMPENSATION_FAILED để đối soát	payment → tuition

Kịch bản demo (5 phút)

1

Đăng nhập

nguyenvana / password123
Số dư 15.000.000

2

Tra cứu

MSSV 524H0024
2 khoản chưa thanh toán

3

Khởi tạo & OTP

Chọn khoản phí → Xác nhận
Xem OTP tại Mailpit :8025

4

Xác thực & thanh toán

Nhập OTP → COMPLETED
Biên lai + email, số dư giảm

5

Concurrency B

2 tài khoản cùng trả 1 khoản
Người thứ hai: 409 FEE_LOCKED

6

Concurrency A & Saga

Số dư chỉ đủ 1 khoản
1 COMPLETED, 1 FAILED + hoàn tác

CHUẨN BỊ TRƯỚC KHI BÁO CÁO

- Chạy: docker compose up -d --build (hoặc start-all.bat)
- Frontend :5173 · Swagger UI :8080 · Mailpit :8025
- DB có sẵn dữ liệu mẫu; mở sẵn webcam, chia sẻ màn hình

DỮ LIỆU MẪU

- Tài khoản (mật khẩu password123): nguyenvana, tranthib, lehoangc, phamminhd, vothue, ngothif
- MSSV: 524H0024, 524H0129, 524H0041
- Test tự động: Backend/test-payment.js (CONCURRENCY A & B)

Hỏi đáp: các quyết định thiết kế

Q1

Vì sao 1 MySQL server, 4 database riêng?

Giữ ranh giới dữ liệu (mỗi service một MySQL user, không truy vấn chéo bảng) mà vẫn nhẹ để chạy demo trên một máy.

Q2

Không có FK chéo DB thì giữ toàn vẹn thế nào?

Kiểm tra qua API, FSM + Saga, khóa dòng/cập nhật có điều kiện ngay trong service sở hữu dữ liệu; transactions lưu snapshot.

Q3

Vì sao Payment là Orchestrator?

Một nơi điều phối FSM, Saga, truy vết và bù hoàn; các service còn lại không gọi nhau nên đồ thị phụ thuộc đơn giản.

Q4

Vì sao gạch nợ trước, trừ tiền sau?

Giữ khoản phí trước; trừ tiền lỗi thì revert-paid (retry 3 lần). Hoàn tác lỗi → COMPENSATION_FAILED, đối soát thủ công.

Q5

Nhập sai OTP có được thử lại?

Không: sai hoặc hết hạn thì giao dịch FAILED, vừa chống dò OTP vừa đơn giản hóa FSM; phải tạo giao dịch mới.

Q6

Mất mạng hoặc bấm hai lần thì sao?

Idempotency-Key + UNIQUE (user_id, key): gửi lại cùng key nhận lại đúng giao dịch cũ, không tạo giao dịch hay OTP mới.